

Exception handling and multithreading



UNIT – III EXCEPTION HANDLING AND MULTITHREADING 8L, 12P

Exception Handling – Java's Built-in Exceptions – User defined Exception - Assertions.

Multithreading- Priorities-Synchronization – Avoiding Deadlocks- Wrappers- Autoboxing and Unboxing.

explain the concepts with code + theory
don't be beginners

Excellent — since you don't want a beginner-level explanation, let's cover **Unit III: Exception Handling and Multithreading in Java** in a **concise yet technically deep** manner, including **concepts + well-commented code examples**.

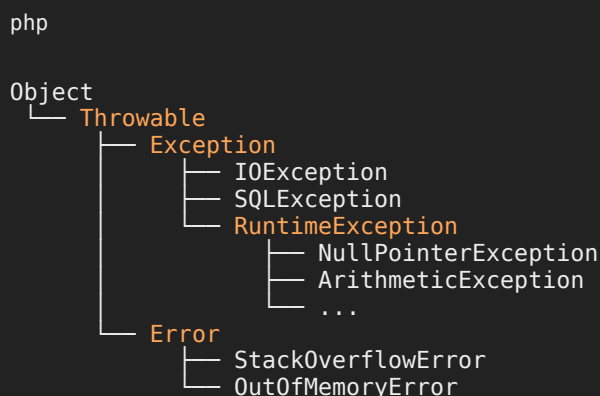
⚙️ 1. Exception Handling

◆ Concept

Exception handling provides a mechanism to separate **error-handling logic** from **normal program flow**. Java exceptions are **objects** that derive from `Throwable`. The two main subclasses are:

- **Checked Exceptions** → Subclasses of `Exception` (compile-time)
- **Unchecked Exceptions** → Subclasses of `RuntimeException` (run-time)
- **Errors** → Indicate serious problems like `OutOfMemoryError`, not meant to be handled by applications

◆ Hierarchy



◆ Built-in Exception Example

```

java

class BuiltInExceptionDemo {
    public static void main(String[] args) {

```

```
try {
    int a = 10, b = 0;
    int result = a / b; // ArithmeticException
    System.out.println(result);
} catch (ArithmeticException e) {
    System.err.println("Error: Division by zero not allowed.");
    System.err.println("Exception details: " + e);
} finally {
    System.out.println("Cleanup code executed.");
}
}
```

Notes:

- `try` block encloses risky code.
- `catch` handles the exception.
- `finally` executes regardless of exception outcome.
- You can rethrow an exception or wrap it inside a custom one for better abstraction.

◆ User-Defined Exception

Custom exceptions let you model **domain-specific error conditions**.

```
java

class InvalidAgeException extends Exception {
    public InvalidAgeException(String msg) {
        super(msg);
    }
}

class CustomExceptionDemo {
    static void register(int age) throws InvalidAgeException {
        if (age < 18)
            throw new InvalidAgeException("Age below 18 not allowed for registration.");
        System.out.println("Registration successful for age: " + age);
    }

    public static void main(String[] args) {
        try {
            register(15);
        } catch (InvalidAgeException e) {
            e.printStackTrace();
        }
    }
}
```

Advanced tip:

Throwing custom exceptions improves maintainability and testability. Always extend from `Exception` or `RuntimeException` depending on whether you want it checked or unchecked.

◆ Assertions

Assertions are used for **internal consistency checks**, not for user input validation.

Enable at runtime using:

```
java -ea ClassName
```

```

java

class AssertionDemo {
    public static void main(String[] args) {
        int x = 10;
        assert x > 0 : "x must be positive";
        System.out.println("Assertion passed");
    }
}

```

Use case: runtime sanity checks during development/testing (e.g., invariants in algorithms, loop correctness).

2. Multithreading

◆ Concept

Java provides **true concurrent execution** using threads.
A thread is a **lightweight process** managed by the JVM.

Ways to create threads:

1. Extend `Thread`
2. Implement `Runnable`
3. Use `ExecutorService` (preferred in production-grade code)

◆ Example: Thread Creation via Runnable

```

java

class Worker implements Runnable {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(Thread.currentThread().getName() + " -> " + i);
            try { Thread.sleep(500); } catch (InterruptedException ignored) {}
        }
    }
}

public class RunnableDemo {
    public static void main(String[] args) {
        Thread t1 = new Thread(new Worker(), "Worker-1");
        Thread t2 = new Thread(new Worker(), "Worker-2");

        t1.start();
        t2.start();
    }
}

```

Notes:

- `start()` spawns a new thread.
- `run()` defines thread behavior.
- `Thread.sleep()` simulates delay; throws `InterruptedException`.

◆ Thread Priorities

Threads have priority between 1–10.

`Thread.MIN_PRIORITY` (1) → lowest

`Thread.NORM_PRIORITY` (5) → default

`Thread.MAX_PRIORITY` (10) → highest

```
java

class PriorityDemo extends Thread {
    public void run() {
        System.out.println(getName() + " running with priority " + getPriority());
    }

    public static void main(String[] args) {
        PriorityDemo t1 = new PriorityDemo();
        PriorityDemo t2 = new PriorityDemo();
        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.MAX_PRIORITY);
        t1.start();
        t2.start();
    }
}
```

Note: Priority doesn't guarantee execution order — it's *scheduler-dependent*.

◆ Synchronization

To prevent **race conditions**, Java provides the `synchronized` keyword.

```
java

class Counter {
    private int count = 0;
    public synchronized void increment() { count++; }
    public int getCount() { return count; }
}

public class SyncDemo {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();

        Runnable task = () -> {
            for (int i = 0; i < 1000; i++) counter.increment();
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);
        t1.start(); t2.start();
        t1.join(); t2.join();

        System.out.println("Final count: " + counter.getCount());
    }
}
```

Concept: Synchronization ensures **mutual exclusion** and **visibility** across threads.

◆ Avoiding Deadlocks

A **deadlock** occurs when multiple threads hold locks and wait on each other.

Avoid by:

- Acquiring locks in a fixed global order
- Using `tryLock()` from `java.util.concurrent.locks.Lock`
- Minimizing synchronized sections

```
java

class Resource {
    final String name;
    Resource(String name) { this.name = name; }
}

class DeadlockFree {
    public static void main(String[] args) {
        final Resource r1 = new Resource("R1");
        final Resource r2 = new Resource("R2");

        Thread t1 = new Thread(() -> {
            synchronized (r1) {
                System.out.println("T1 locked " + r1.name);
                synchronized (r2) {
                    System.out.println("T1 locked " + r2.name);
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (r1) { // Always lock in same order (r1→r2)
                System.out.println("T2 locked " + r1.name);
                synchronized (r2) {
                    System.out.println("T2 locked " + r2.name);
                }
            }
        });

        t1.start();
        t2.start();
    }
}
```

◆ Wrappers, Autoboxing, and Unboxing

Wrapper classes are object representations of primitive types (`Integer`, `Double`, etc.).

They enable use in collections, generics, and APIs requiring objects.

Autoboxing: automatic conversion from primitive → wrapper

Unboxing: automatic conversion from wrapper → primitive

```
java

public class WrapperDemo {
    public static void main(String[] args) {
        Integer x = 10;           // autoboxing
        int y = x;                 // unboxing
        System.out.println(x + y);
    }
}
```

Advanced note: excessive autoboxing can hurt performance in tight loops — prefer primitives in high-throughput code.

Summary Table

Concept	Key Mechanism	Example
Exception Handling	try-catch-finally, throw, throws	Division by zero
Custom Exception	Extend Exception	InvalidAgeException
Assertions	assert keyword	Debug validation
Multithreading	Runnable, Thread	Concurrent print
Synchronization	synchronized , Lock	Counter increment
Avoid Deadlocks	Lock ordering, tryLock	Resource lock order
Wrapper Classes	Primitive ↔ Object	Integer, Double
Autoboxing / Unboxing	Automatic conversion	Integer i = 5; int x = i;

Would you like me to add a **mini advanced project example** (like a multithreaded logging system or producer-consumer with synchronization and custom exceptions) that ties all these together for exam + interview prep?